

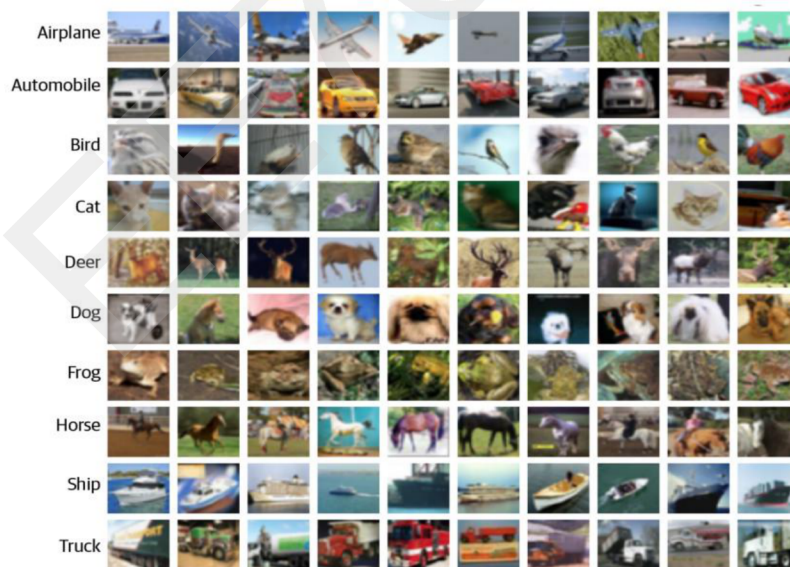
EE708: Fundamentals of Data Science and Machine Intelligence

Rajesh M Hegde
Dept. of EE IIT Kanpur



Module 7A Deep Feedforward Networks, Regularization, and Optimization for Training Deep Models

Deep Learning Frameworks : DNN for recognizing CIFAR10 Data

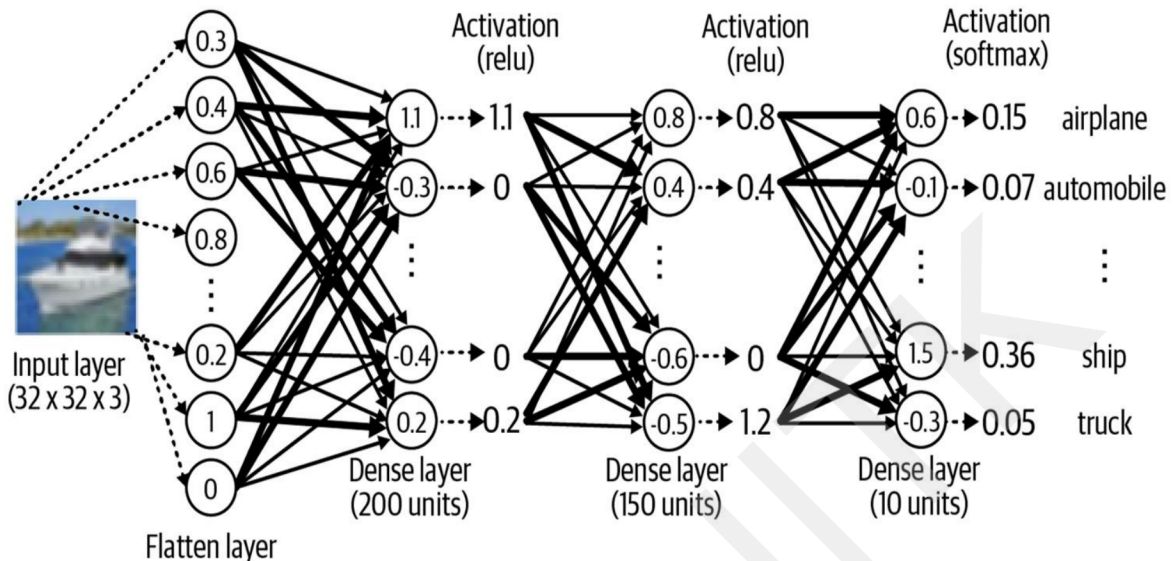


EE708

Example images from the CIFAR-10 dataset.

2

Deep Learning Frameworks : DNN for recognizing CIFAR10 Data



EE708

Module 7

3

Deep Learning Frameworks : DNN for recognizing MNIST Data

- The MNIST dataset
- Contains 60,000 training images and 10,000 test images.
- Dataset consists of labels that describe which digit each image represents.
- Original images are 28x28 pixels.
- Each pixel is represented by a grayscale value ranging from 0 to 255.



Images from the MNIST dataset.

Dataset source: Employees at the American Census Bureau and American high school students. The dataset was made available in 1998 (LeCun, Bottou, Bengio, et al., 1998).

EE708

Module 7

4

Training Set, Test Set and Generalization

- Why to build a neural network to predicts the output for a set of labeled examples?
 - It would be simpler to just create a lookup table based on all the training examples.
 - This brings us to the concept of generalization.
- The goal of an ML model is not just to provide correct predictions for data that it has been trained on; the more important goal is to provide correct predictions for previously unseen data.
- Therefore, the dataset is divided into a training dataset and a test dataset.
 - Training dataset is used to train the model
 - Test dataset is used to evaluate how well the model could generalize to previously unseen data.
- If a model performs well on the training set but poorly on the test set, it indicates that it memorized the training data instead of learning the underlying patterns for generalization.

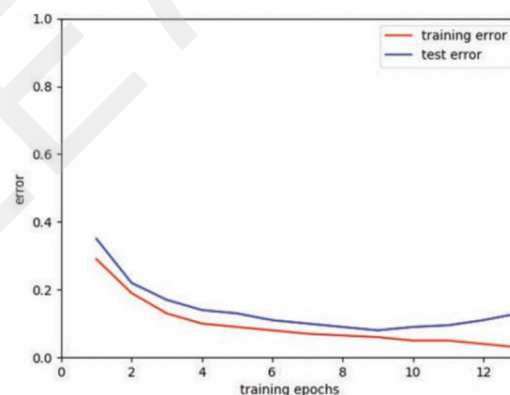
EE708

Module 7

5

Training set, Test set and Generalization

- Monitor the training error and test error during training to establish whether the model is learning to generalize.



How training error and test errors can evolve during the learning process

EE708

Module 7

6

Hyperparameter Tuning

- Hyperparameters are parameters adjusted by the learning algorithm.
- Learning rate, network topology, and activation function type are hyperparameters.
- Using a validation set for hyperparameter tuning is a crucial concept.

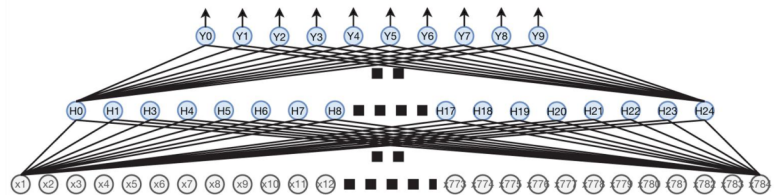
Training and Inference

- Experiments and discussions have focused on the training process of the network.
- Testing is interleaved during training to evaluate the network's learning progress.
- Training: The process of determining the network's weights, typically done before deployment.
- In production, the network is often used solely for inference.

How to decide the structure of DNN for Digit Classification

- Say Each image contains 784 (28×28) pixels, so the network has **784 input nodes**.

- **Inputs** are fed to a hidden layer with **25 (arbitrarily chosen) neurons**.



Network architecture for digit classification.

- **The hidden layer** feeds the output layer with **ten neurons**, one for each digit to recognize.
- **Activation function**: **tanh** for the hidden neurons and the **logistic** sigmoid for the output layer.
- **Network is fully connected**: that is, each neuron in one layer connects to all neurons in the next layer.

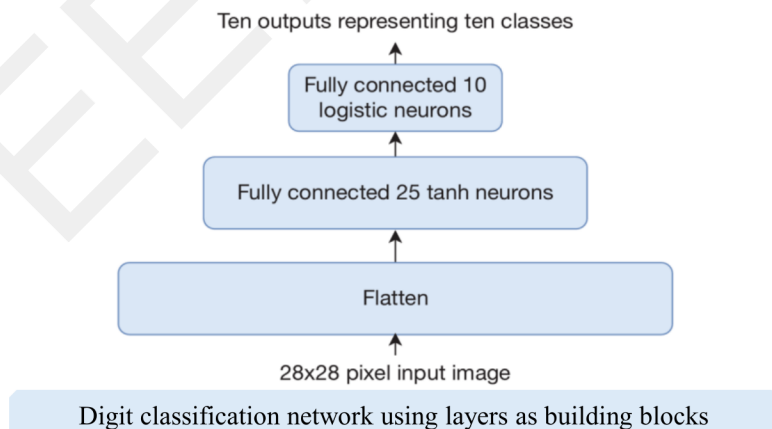
EE708

Module 7

9

Structure of DNN for Digit Classification

Rectangular boxes with rounded corners depict a layer of neurons, as opposed to circles that represent individual neurons.



Digit classification network using layers as building blocks

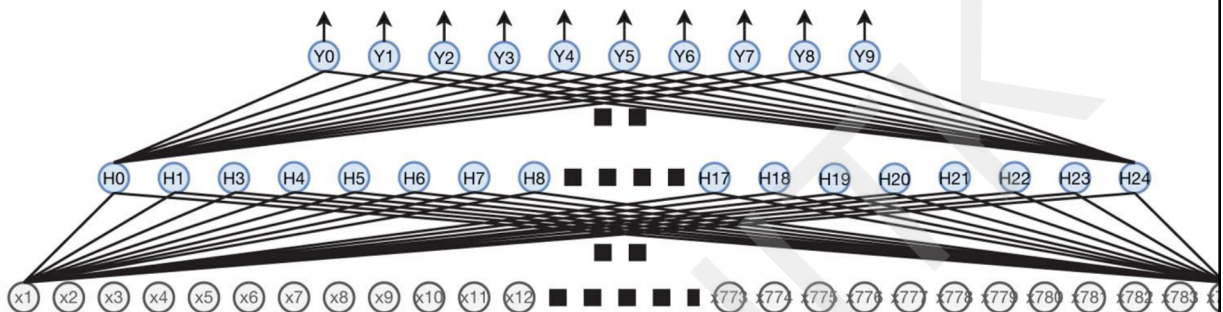
Digit recognition network at this higher abstraction level

EE708

10

Structure of DNN for Digit Classification

- The arrangement as a 1D vector may seem to lose the spatial relationship between neighboring pixels.
- Neighboring pixels in the y-Direction seems to be separated by 28 input neurons.
- In fully connected networks, all pixels are connected to all hidden neurons, meaning no pixels are “separated”- all are equally accessible to each neuron.



EE708

Module 7

11

Calculating Loss Function for Multiclass Classification

- For a network with C outputs and single training example, loss (error) function is defined as the sum of the squared error for each individual output.

$$Error = \sum_{j=0}^{C-1} (y_j - \hat{y}_j)^2$$

where $\hat{y}_j$ refers to the output value of neuron Y_j

- For N training samples

$$Error = \frac{1}{N} \sum_{i=0}^{N-1} \sum_{j=0}^{C-1} (y_j^{(i)} - \hat{y}_j^{(i)})^2$$

- To simplify derivative later, loss function is scaled by factor of 0.5

$$Error: e(\hat{\mathbf{y}}) = \frac{1}{2} \sum_{j=0}^{C-1} (y_j - \hat{y}_j)^2$$

EE708

Module 7

12

Computing the Loss Function for Multiclass Classification

- Compute the error term for each of the C output neurons.
- Apply backpropagation algorithm:

- The error term for neuron Y_1 with output value $\hat{y}_1$:

$$\frac{\partial e}{\partial \hat{y}_1} = \sum_{j=0}^{C-1} \frac{\partial}{\partial \hat{y}_1} \frac{(y_j - \hat{y}_j)^2}{2} = \frac{2(y_1 - \hat{y}_1)}{2} \cdot (-1) = -(y_1 - \hat{y}_1)$$

- Similarly, The error term for neuron Y_2 is $-(y_2 - \hat{y}_2)$
- In the general case, the error term for Y_j is $-(y_j - \hat{y}_j)$

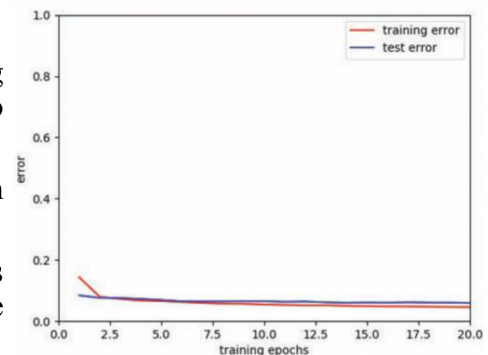
EE708

Module 7

13

Interpreting Loss Function for Multiclass Classification

- Example : Digit Classification
- Both the training and the test error are decreasing over time, and the test error does not yet start to increase at the right side of the chart.
- Does not seem to have a significant problem with overfitting.
- The training error is lower than the test error, this is common and not a reason for concern as long as the gap is not too big.



Training and test error when learning to classify digits

EE708

Module 7

14

Gradient Descent Types : Batch, Mini Batch, Hybrid, and Stochastic

- Batch Gradient Descent
 - Uses the entire training dataset for each gradient update.
- Mini-Batch Gradient Descent
 - Smaller sizes - Mini batch size is another tunable hyperparameter that typically ranges between 32 and 256.
- Hybrid : A hybrid approach using subsets (mini-batches) of the training data; often referred to by its *batch size*.
- Stochastic Gradient Descent (SGD)
 - Uses a single training example for each gradient update but is sometimes used to describe mini-batch gradient descent.
 - SGD is a mini-batch size of 1 and it is sometimes called *online learning* as it can operate in real-time settings.

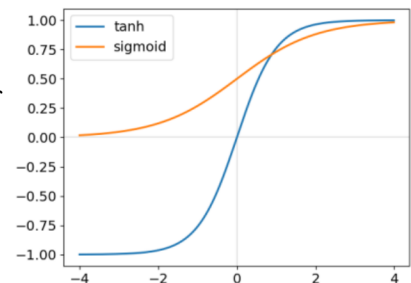
EE708

Module 7

15

Issues with Neural Network Training

- Common issues occur due to Act. Function and its gradient
 1. Saturated Neurons
 - Occurs when the output of a neuron is stuck at one extreme of the activation function.
 - Results from the neuron when the input is too large or too small and the output at the extreme ends of the function's range (close to 0 or 1 for sigmoid, or -1 or 1 for tanh).
 2. Vanishing Gradient
 - Occurs during backpropagation when gradients get smaller and smaller as they propagate backward through the network.
 - Results from having activation functions that squash outputs to a small range (like sigmoid or tanh) or deep layers with small gradients.
 3. Overfitting
 - Occurs when a model learns the training data too well, leading to poor performance on test data.



Saturated neurons and vanishing gradient problems affect the learning process and can lead to slower or failed convergence.

EE708

Module 7

16

Saturated Neurons

- Occurs when the output of a neuron is stuck at saturated region (extreme ends) of the activation function.
- Techniques to **avoid** saturated neurons:
 - Weight initialization
 - Input standardization
- Techniques to **mitigate** effect of saturated output neurons:
 - Cross-entropy loss function

EE708

Module 7

17

Techniques to Avoid Saturated Neurons

1. Weight Initialization

Each neuron computes $z = w_1x_1 + w_2x_2 + \dots + w_nx_n$

- With many inputs, large weights make the summed input z very large, pushing activation functions into saturation.
- Initializing smaller weights keep z near to zero, reducing the risk of saturation
- Two popular weight initialization strategies are:
 - Glorot initialization (*Glorot and Bengio, 2010*)
 - Recommended for tanh- and sigmoid-based neurons
 - He initialization (*He et al., 2015b*)
 - Recommended for ReLU-based neurons
- Both of these take the **number of inputs** into account, and **Glorot initialization also takes the number of outputs into account.**
- Both come in **two types**: one based on a **uniform random distribution** and the other on a **normal random distribution.**

EE708

Module 7

18

Techniques to Avoid Saturated Neurons

Glorot initialization	He initialization
To ensure that the variance of activations and gradients remains consistent across layers.	- ReLU tends to zero out negative inputs, which can lead to a loss of variance in the network. - To address this, the variance of the weights is set to be twice as large as Glorot Initialization.
Recommended for tanh- and sigmoid-based neurons.	Recommended for ReLU-based neurons.
Normal distribution: $\mathbf{W} \sim \mathcal{N}\left(0, \frac{1}{N_{\text{avg}}}\right)$	Normal distribution: $\mathbf{W} \sim \mathcal{N}\left(0, \frac{2}{N_{\text{in}}}\right)$
Uniform distribution: $\mathbf{W} \sim \mathcal{U}\left(-\sqrt{\frac{3}{N_{\text{avg}}}}, \sqrt{\frac{3}{N_{\text{avg}}}}\right)$	Uniform distribution: $\mathbf{W} \sim \mathcal{U}\left(-\sqrt{\frac{6}{N_{\text{in}}}}, \sqrt{\frac{6}{N_{\text{in}}}}\right)$
$N_{\text{avg}} = \frac{N_{\text{in}} + N_{\text{out}}}{2}$, where N_{in} and N_{out} are number of input and output neurons of a layer.	

EE708

Module 7

19

Techniques to Avoid Saturated Neurons

2. Input Standardization

- In Addition to initializing the weights properly, it is important to preprocess the input data.
- Standardizing the input data to be centered around 0 and with most values close to 0 reduces the risk of saturating neurons from the start.
- Subtract the mean and divide by the standard deviation, avoids neuron input that can lead to saturation. (**Mu – Sigma Normalization**)
- Batch Normalization** (*Ioffe and Szegedy, 2015*)
 - Normalize values inside of the network to prevent hidden neurons from becoming saturated.
 - Two main ways to apply batch normalization:
 - Apply the normalization on the input to the activation function (after the weighted sum). (*as in original paper*)
 - The activation function is applied to the output of the batch normalization operation.
 - Apply the normalization on the output of the activation function.

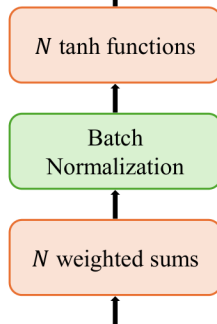
EE708

Module 7

20

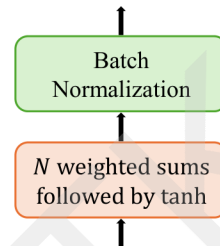
Techniques to Avoid Saturated Neurons

Batch normalization before
activation (original
version)



Batch normalization as presented by Ioffe and Szegedy (2015)

Batch normalization after
activation



Batch normalization is applied to the output of the activation functions.

EE708

Module 7

21

Techniques to Mitigate Effect of Saturated Output Neurons

1. Cross-Entropy Loss Function

- Saturation can occur when the output neuron is pushed toward values of 0 or 1, leading to its activation reaching a saturated state.
- *LeCun, Bottou, Orr, and Müller (1998)* proposed setting the desired output to 0.1 or 0.9 to prevent neurons from entering deep saturation regions.

EE708

Module 7

22

Techniques to Mitigate Effect of Saturated Output Neurons

1. Cross-Entropy Loss Function

To understand effect of loss function, revisit backpropagation algorithm:

Mean Square Error: where $\hat{y} = S(z) = \frac{1}{1+e^{-z}}$

$$e(\hat{y}) = \frac{(y - \hat{y})^2}{2}$$

- Derivative of Error (MSE)

$$\frac{\partial e}{\partial \hat{y}} = -(y - \hat{y})$$

- Derivative of Sigmoid

$$\frac{\partial \hat{y}}{\partial z} = S'(z) = S(z) \cdot (1 - S(z)) = \hat{y} \cdot (1 - \hat{y})$$

- Error gradient term for the output neuron:

$$\frac{\partial e}{\partial z} = \frac{\partial e}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = -(y - \hat{y}) \cdot \hat{y} \cdot (1 - \hat{y})$$

If $S'(z)$ is close to 0, then saturated neurons/no error will backpropagate through the network.

EE708

Module 7

23

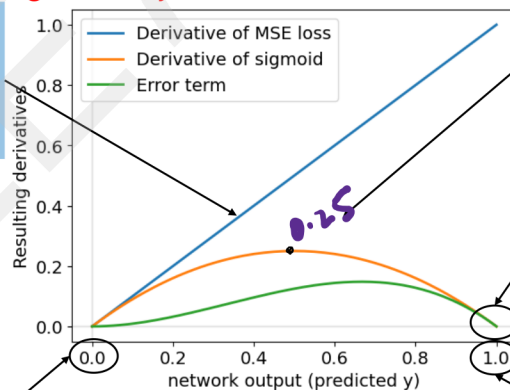
Techniques to Mitigate Effect of Saturated Output Neurons: MSE

Plot of the error function, the derivative of the logistic sigmoid function, and derivative of the **MSE loss function**, when target value, $y = 0$.

Derivative of MSE:

$$\frac{\partial e}{\partial \hat{y}} = -(y - \hat{y}) = \hat{y}$$

Increases as the output moves away from the target value.



Network output matches ground truth (output value results from a weighted sum $z \ll 0$)

Output neuron error gradient term (green curve) is well behaved in the range where network output matches ground truth and up to a point where it is moderately far away.

Derivative of sigmoid:

$$\frac{\partial \hat{y}}{\partial z} = S'(z) = \hat{y} \cdot (1 - \hat{y})$$

Initially increases but then decreases as the output moves away from the target value.

Error gradient term:

$$\frac{\partial e}{\partial z} = \frac{\partial e}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \hat{y}^2(1 - \hat{y})$$

Increases as output moves away from the target value but then decreases as sigmoid enters saturated region.

Network output is opposite of ground truth (output value results from a weighted sum $z \gg 0$)

EE708

Module 7

24

Techniques to Mitigate Effect of Saturated Output Neurons: Cross Entropy

- The issue arises because the activation function's derivative approaches zero, while the loss function's derivative remains at most 1, resulting in their product approaching zero.
- One potential solution is to use a different loss function whose **derivative can take on much higher values than 1**.

- Consider loss function known as **cross-entropy loss function**: where $\hat{y} = S(z) = \frac{1}{1+e^{-z}}$

$$e(\hat{y}) = -(y \cdot \ln(\hat{y}) + (1 - y) \cdot \ln(1 - \hat{y}))$$

- Derivative of Error (CEL)

$$\frac{\partial e}{\partial \hat{y}} = -\left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right)$$

- Derivative of Sigmoid

$$\frac{\partial \hat{y}}{\partial z} = S'(z) = S(z) \cdot (1 - S(z)) = \hat{y} \cdot (1 - \hat{y})$$

- Error gradient term for the output neuron:

$$\frac{\partial e}{\partial z} = \frac{\partial e}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = -\left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right) \cdot \hat{y} \cdot (1 - \hat{y}) = \hat{y} - y$$

EE708

Module 7

26

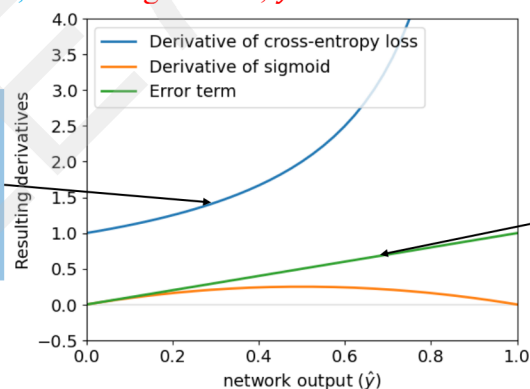
Techniques to Mitigate Effect of Saturated Output Neurons : Cross Entropy

Plot of the Error Term, the derivative of the logistic sigmoid function, and derivative of the **cross-entropy loss function**, when target value, $y = 0$.

Derivative of CEL:

$$\frac{\partial e}{\partial \hat{y}} = -\left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right) = \frac{1}{1-\hat{y}}$$

Increases steeply towards infinity as output moves further away from target value.



Error gradient term:

$$\frac{\partial e}{\partial z} = \frac{\partial e}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \hat{y}$$

Resulting error term for the output neuron is no longer 0 when the output is opposite of target value.

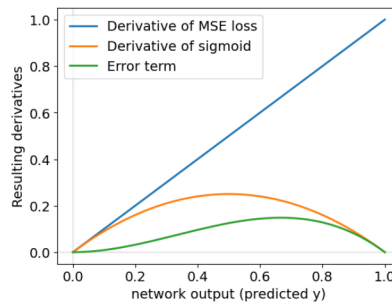
EE708

Module 7

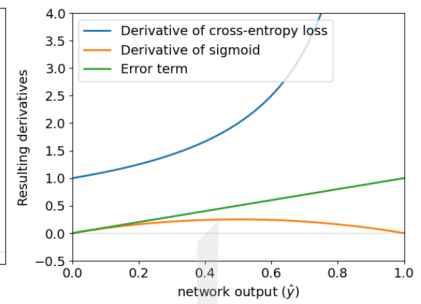
27

Techniques to Mitigate Effect of Saturated Output Neurons : MSE versus Cross Entropy ($y = 0$)

- The y-range of cross-entropy loss is increased compared to MSE, capturing more of the range of the new loss function.
- The derivative of the cross-entropy loss function does increase significantly at the right end and the resulting product (the green line) approaches 1 in the case where the neuron is saturated.
- The backpropagated error is no longer 0, and the weight adjustments will no longer be suppressed.



For mean square error



For cross-entropy loss

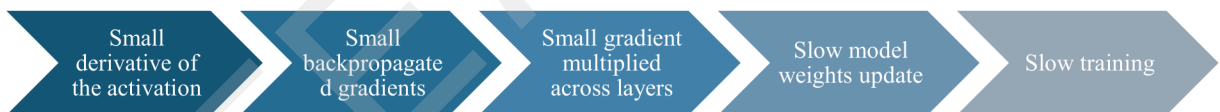
EE708

Module 7

28

Problem of Vanishing Gradient

- Occurs during backpropagation when gradients get smaller and smaller as they propagate backward through the network.



- ReLU (and its variants) are less likely to cause vanishing gradients because their gradient is either 0 (for negative inputs) or constant (for positive inputs).
- Different activation functions can be used for hidden layers to avoid vanishing gradient.

EE708

Module 7

29

Techniques to Avoid Vanishing Gradient

Different Activation Functions for Hidden Layer

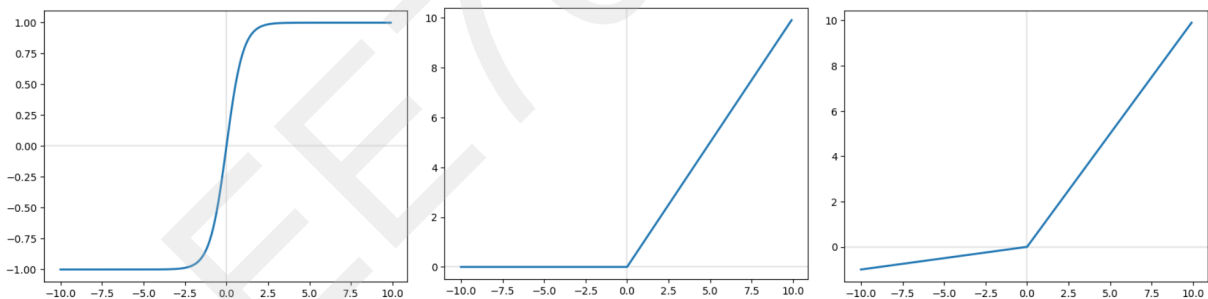
- Choosing a different loss function does not help for the hidden layers.
- Hidden neurons can still be saturated, resulting in derivatives close to 0 and vanishing gradients.
- Right choice of activation function can help.
 - Activation function should be nonlinear.
- Activation functions for hidden layer:
 - All activation functions except for tanh effectively fight vanishing gradients in hidden units.
 - The *tanh*, *ReLU*, *leaky ReLU*, *softplus*, *elu*, and *maxout* functions can all be considered for hidden units, but tanh has a problem with vanishing gradients.

EE708

Module 7

30

Avoid Vanishing Gradient : Activation Functions for Hidden Layer



Hyperbolic Tangent (tanh)

Function

$$\frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Rectified Linear Unit (ReLU)

$\max(0, x)$

Leaky ReLU Function

$\max(0.1x, x)$
 (Xu, Wang, et al., 2015)

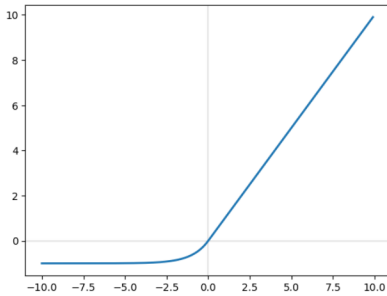
Important activation functions for hidden neurons.

EE708

Module 7

31

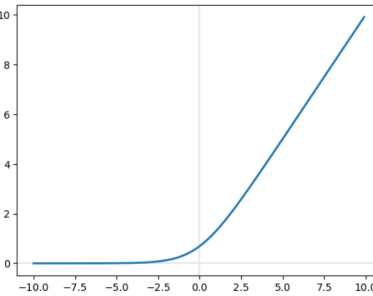
Techniques to Avoid Vanishing Gradient



Exponential linear unit (elu)

$$f(x) = \begin{cases} \alpha(e^x - 1), & x < 0 \\ x, & x \geq 0 \end{cases}$$

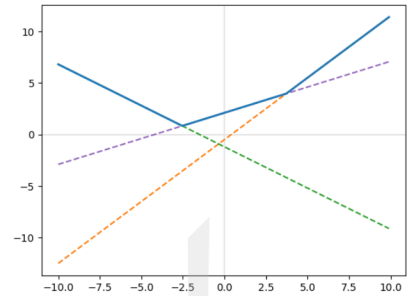
(Shah et al., 2016)



softplus

$$\ln(1 + e^x)$$

(Dugas et al., 2001)



maxout

$$\max(\{w_k x + b_k\}_{k=1}^K)$$

(Goodfellow et al., 2013)

Important activation functions for hidden neurons.

EE708

Module 7

32

Saturated Neurons and Vanishing Gradient : Summary and Inferences

Aspect	Saturated Neurons	Vanishing Gradient
Definition	Neurons are stuck at extreme activation values	Gradients diminish as they backpropagate through layers
Cause	Activation functions (e.g., sigmoid, tanh) squash outputs excessively.	Deep networks and small derivatives of activation functions.
Impact	Neurons stop contributing to learning due to near-zero gradients.	Early layers fail to learn effectively due to tiny weight updates.
Solution	Use ReLU, Leaky ReLU, or proper weight initialization.	Use ReLU, batch normalization, gradient clipping, or better initialization.

EE708

Module 7

33

Overfitting Problem

- Occurs when a model learns the training data too well, leading to poor performance on test data.
- Causes:
 - High model complexity: Too many parameters or deep layers for the given dataset.
 - Insufficient training data: Limited data prevents the model from generalizing.
 - Noisy data: Irregular patterns or errors in the training set get learned.
- Consequences:
 - Poor generalization: Performs well on training data but poorly on test or real-world data.
 - Increased error rates: Predictions on new data are unreliable.
 - High variance: Sensitive to minor changes in the input.
- Solutions:
 1. Increase data
 2. Simplify the model
 3. Use a validation set

EE708

Module 7

34

Technique to Avoid Overfitting

Using a Validation Set

- Overfitting problem can be solved by splitting up the dataset into a training dataset, a validation dataset, and a test dataset.
 - Training dataset is used to train the weights of the model.
 - Validation dataset is used to tune the hyperparameters
 - Testing dataset determines how well the final model works on not-yet-seen data.
- How to decide how much of the original dataset to use as training, validation, and test set:
 - It is determined on a case-by-case basis and depends on the variance in the data distribution.
 - If the variance is low, a smaller fraction is used for validation.
 - If the variance is high, a larger fraction is needed.
 - In the absence of variance information
 - Without a validation set, the training to testing ratio is 70: 30 or 80: 20.
 - With the validation set, the training to validation to testing ratio is 60: 20: 20.

EE708

Module 7

35

Cross-validation to avoid Overfitting and Improve Performance

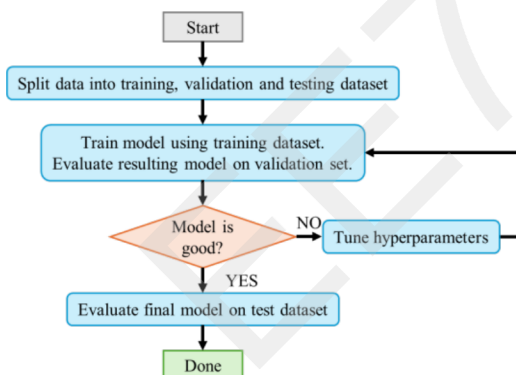
- Using a validation set reduces the available training data that can be problematic with limited data.
- Cross-validation* avoids holding out validation data, though it requires extra computation.
- One of the most popular cross-validation techniques, known **as k-Fold Cross-Validation**:
 - Data is first split into training (e.g., 80%) and test sets (20%).
 - The test set is reserved for final model evaluation and not used for training or tuning.
 - The training set is divided into k equal-sized folds (typically $5 \leq k \leq 10$).
 - $k - 1$ folds are used for training, and 1 fold is used for validation, rotating through all folds.
 - For $k = 5$, this creates 5 training-validation instances:
 - Instance 1: Folds 1 to 4 for training, Fold 5 for validation.
 - Instance 2: Folds 1 to 3, 5 for training, Fold 4 for validation, and so on.
- Cross-validation maximizes data utilization for training and provides robust validation.

EE708

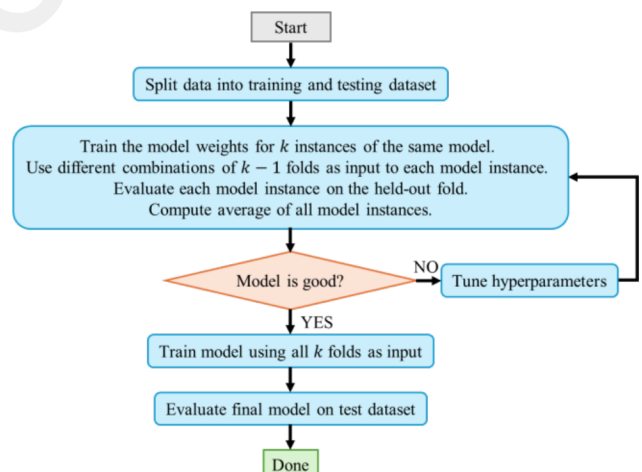
Module 7

36

Cross-validation to avoid Overfitting and Improve Performance



Learning algorithm with training, validation and testing dataset.



Learning algorithm with training and testing dataset using **k-fold cross-validation**.

EE708

Module 7

37

Dropout to avoid Overfitting and Improve Performance

- Dropout is a technique meant to prevent overfitting the training data by dropping out units in a neural network.
 - Neurons are either dropped with probability p or kept with probability $1 - p$.

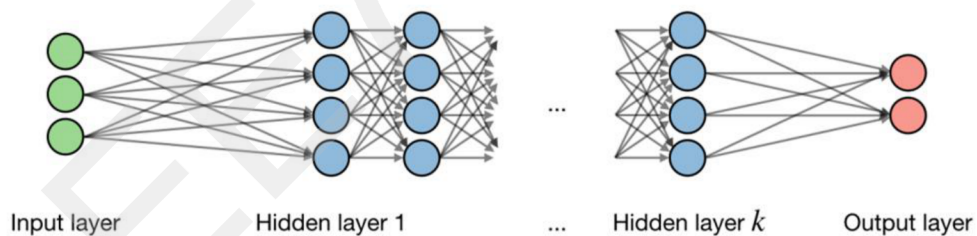
EE708

Module 7

38

Summary of DNN

- The vocabulary around neural network architectures is described in the figure:



- By noting i the i th layer of the network and j the j th hidden unit of the layer, we have:

$$z_j^{[i]} = w_j^{[i]T} x + b_j^{[i]}$$

where w , b , z are the weight, bias, and output respectively.

EE708

Module 7

39

Summary of DNN

- Cross-entropy loss is commonly used and defined as:

$$L(z, y) = -[y \log(z) + (1 - y) \log(1 - z)]$$

- The learning rate (η) indicates the pace of weight update.
 - It can be fixed or adaptively changed.
 - The most popular optimizer is Adam, a method that adapts the learning rate.
- Backpropagation is a method to update the weights in the neural network by considering the actual output and the desired output.
 - The derivative with respect to weight w is computed using the chain rule.
- Dropout is a technique meant to prevent overfitting the training data by dropping out units in a neural network.
 - Neurons are either dropped with probability p or kept with probability $1 - p$.

EE708

Module 7

40

Summary of DNN

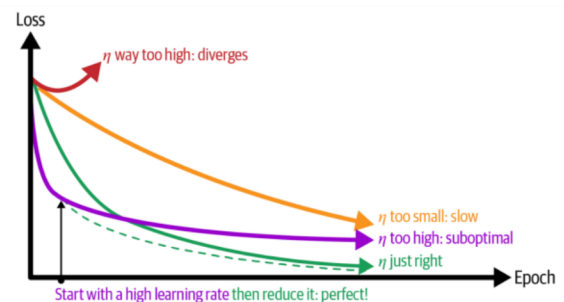
- In a neural network, weights are updated as follows:

- Take a batch of training data.
- Perform forward propagation to obtain the corresponding loss.

$$L(z, y) = -[y \log(z) + (1 - y) \log(1 - z)]$$
- Backpropagate the loss to get the gradients.

$$\frac{\partial L(z, y)}{\partial w} = \frac{\partial L(z, y)}{\partial a} \times \frac{\partial a}{\partial z} \times \frac{\partial z}{\partial w}$$
- Use the gradients to update the weights of the network.

$$w \leftarrow w - \eta \frac{\partial L(z, y)}{\partial w}$$



Learning curves for various learning rates η .

EE708

Module 7

41